

PROGRAMAÇÃO DE DISPOSITIVOS MÓVEIS

Prof. Tiago Piperno Bonetti
bonetti@prof.unipar.br



Visualizando propriedades da rota

UserForm.js

```
4 export default (props) => {  
5   //visualiza os parâmetros da rota de edição  
6   if (props.route.params) {  
7     console.warn(Object.keys(props));  
8     console.warn(Object.keys(props.route.params));  
9   }
```

UserList.js

```
<Button  
  //acessando a edição passando o usuário selecionado  
  onPress={() => props.navigation.navigate('UserForm', user)}  
  type="clear"  
  icon={<Icon name="edit" size={25} color="#3366cc" />}  
/>
```

PROBLEMS

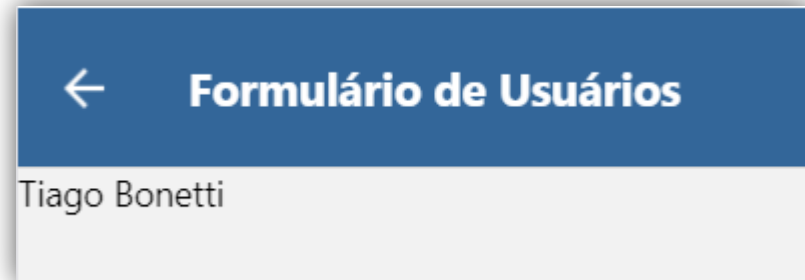
LOGS

Chrome: ▶ ["navigation", "route"]

Chrome: ▶ ["id", "name", "email", "avatarUrl"]

Acessando os dados do usuário para edição

```
1 import React, { useState } from 'react';
2 import { Text } from 'react-native';
3
4 export default (props) => {
5   // Recebendo os dados do usuário via rota
6   // usa os parâmetros como estado inicial
7   // se não tiver, cria um objeto vazio
8   const [user, setUser] = useState(
9     props.route.params ? props.route.params : {}
10  );
11
12   return <Text>{user.name}</Text>;
13 };
```



Criando o form

```
12  return (  
13    <View>  
14      <Text>Nome</Text>  
15      <TextInput  
16        //função para atualizar o usuário  
17        // setUser recebe o usuário atual (... = Spread)  
18        // e sobrescreve o nome  
19        onChangeText={(name) => setUser({ ...user, name })}  
20        placeholder="Informe o nome"  
21        value={user.name}  
22      />  
23    </View>  
24  );
```

Criando o form e estilizando

```
12  return (  
13    <View style={style.form}>  
14  
15      <Text>Nome</Text>  
16      <TextInput  
17        style={style.input}  
18        onChangeText={(name) => setUser({ ...user, name })}  
19        placeholder="Informe o nome"  
20        value={user.name}  
21      />  
22  
23      <Text>E-mail</Text>  
24      <TextInput  
25        style={style.input}  
26        onChangeText={(email) => setUser({ ...user, email })}  
27        placeholder="Informe o e-mail"  
28        value={user.email}  
29      />  
    </View>  
  )
```

```
const style = StyleSheet.create({  
  form: {  
    padding: 10  
  },  
  input: {  
    height: 40,  
    borderWidth: 1,  
    marginBottom: 12,  
    marginTop: 5,  
    padding: 10,  
  }  
})
```

Context API

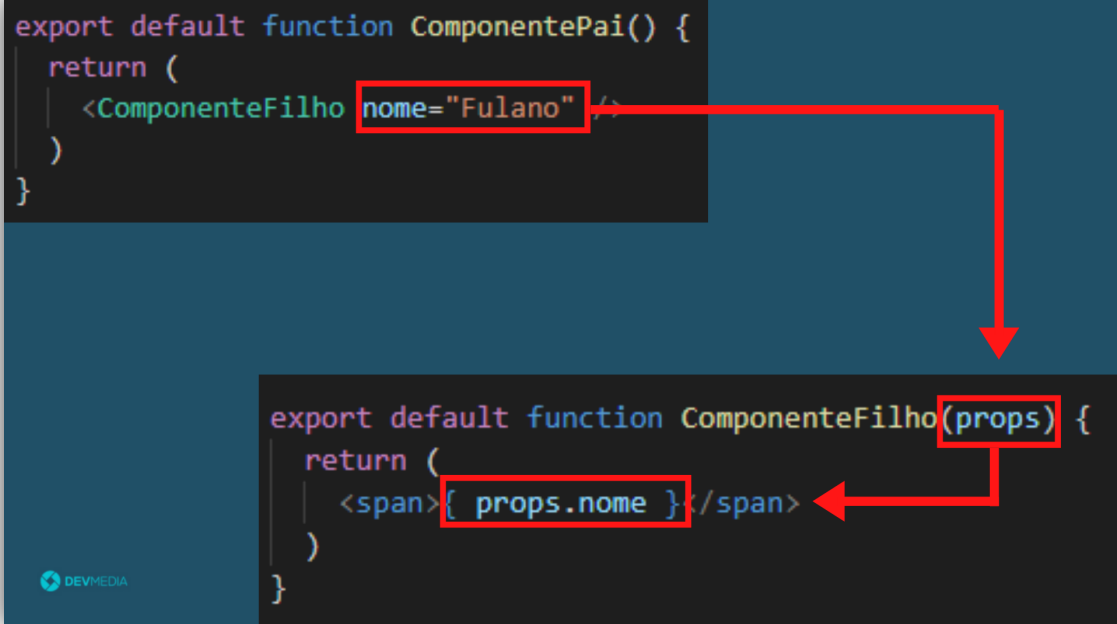
Contexto (context) disponibiliza uma forma de **passar dados** entre a **árvore de componentes** sem precisar passar props manualmente em cada nível.

Em uma aplicação típica do React, os dados são passados de cima para baixo (de pai para filho) via props, mas esse uso pode ser complicado para certos tipos de props.

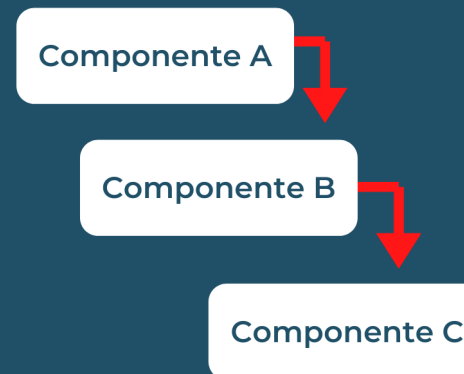
Contexto (context) fornece a forma de compartilhar dados como esses, entre todos componentes da mesma árvore de componentes, sem precisar passar explicitamente props entre cada nível.

Context API

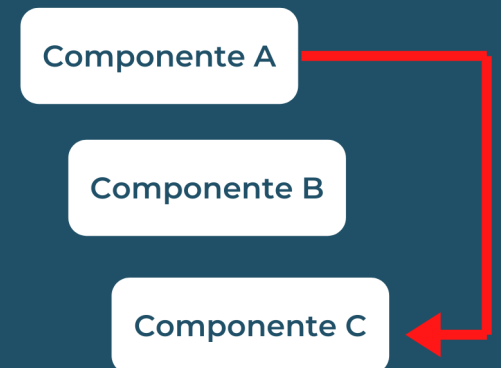
```
export default function ComponentePai() {  
  return (  
    <ComponenteFilho nome="Fulano" />  
  )  
}  
  
export default function ComponenteFilho(props) {  
  return (  
    <span>{ props.nome }</span>  
  )  
}
```



Usando **props**

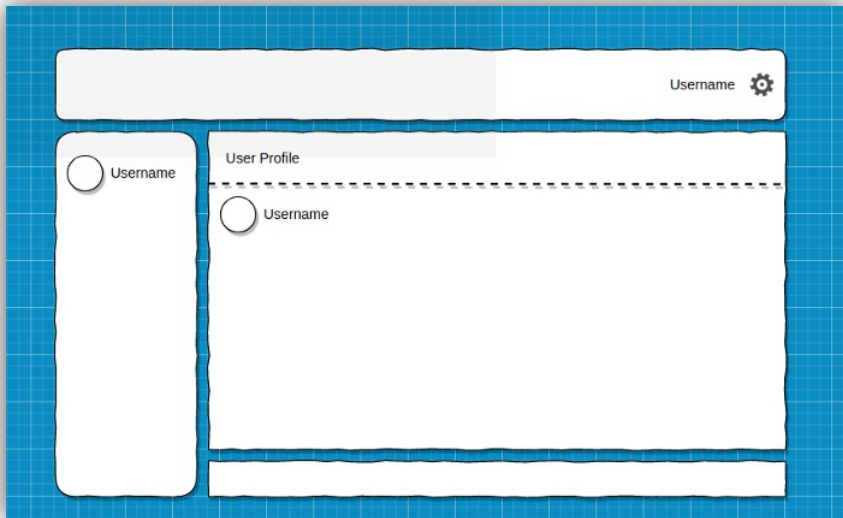


Usando **context** API



Context API - Exemplo

Imagine que temos um dado que deve ser compartilhado entre mais componentes. Exemplo: os dados de um usuário.



Hierarquia e organização dos componentes dessa tela:

```
- body (div#app)
  - app
    - header
      - settings (vai usar a propriedade user)
    - sidebar
      - sidebar-header
        - user-avatar (vai usar a propriedade user)
    - content
      - content-header
      - content-body
        - user-avatar (vai usar a propriedade user)
    - footer
```


Context API

React.createContext

```
const MyContext = React.createContext(defaultValue);
```

Cria um objeto Contexto (context). Quando o React renderiza um componente que assina este objeto Contexto (context), este vai ler o valor atual do **Provider** superior na árvore que estiver mais próximo.

Context.Provider

```
<MyContext.Provider value={/* some value */}>
```

Cada objeto Contexto (context) vem com um componente Provider que permite componentes consumidores a assinarem mudanças no contexto.

Criando o contexto

```
1  import React, { createContext } from 'react'
2  // importando os usuários
3  import users from '../data/users'
4
5  // criando o contexto
6  const UserContext = createContext( { } )
7
8  //exportando o provider
9  export const UsersProvider = (props) => {
10
11    // retorna
12    // value = primeira chave js, segunda objeto
13    return (
14      <UserContext.Provider value={{
15        //objeto state
16        state: {
17          users
18        }
19      }}>
20        {props.children}
21      </UserContext.Provider>
22    )
23  }
24
25  // exportando o contexto
26  export default UserContext
```

Adicionando o contexto na aplicação

```
8  import { UsersProvider } from './context/UserContext';
9
10 //importando as telas
11 import UserList from './views/UserList';
12 import UserForm from './views/UserForm';
13
14 const Stack = createStackNavigator();
15
16 export default function App() {
17   return (
18     <UsersProvider>
19       <NavigationContainer>
20         <Stack.Navigator>
21           initialRouteName="UserList"
```

Usando o contexto na aplicação

```
5 import UsersContext from '../context/UserContext'
6
7 export default (props) => {
8
9   //acessando o contexto
10  const { state } = useContext(UsersContext)
11
12  //função que renderiza cada um dos usuários
13  function getUserItem({ item: user }) {
```

```
<View>
  <FlatList
    keyExtractor={(user) => user.id}
    //usando o context api para acessar a lista de usuários
    data={state.users}
    renderItem={getUserItem}
  />
</View>
```

Referências

Construa um Cadastro COMPLETO em React Native - Com Hooks e Context API: <https://youtu.be/V-uYjDnuXkU>

Escudelario, Bruna de Freitas; Pinho, Diego Martins. **React Native: Desenvolvimento de aplicativos mobile com React**. Casa do código.

Documentação React Native. Disponível em: <https://reactnative.dev/docs>.

Documentação React Navigation. Disponível em:
<https://reactnavigation.org/docs/getting-started>

<https://oblador.github.io/react-native-vector-icons/>